Study Report: Computer Programming for Beginners - 4 Books in 1 (Linux Command-Line + Python)

Written by Gagan Pasupuleti

Family:	Multi-Language Programming
Level:	Beginner
Chapters:	6
Source:	COMPUTER PROGRAMMING FOR BEGINNERS - 4 Books in 1. LINUX COMMAND-LINE + PYTHON.pdf

Summary

This report covers a four-in-one bundle that pairs the Linux command line with Python programming. It teaches navigating and managing files from the terminal, useful command-line tools, then core Python: setup, variables and data types, control flow, functions, and combining shell skills with Python scripts. It is aimed at beginners who want both system skills and programming skills together.

Chapters

Chapter 1: Getting Started with the Linux Command Line

Chapter focus: Getting Started with the Linux Command Line

Pipes chain commands: cat file | grep error sends output to the next tool. chmod +x script makes it executable.

Code Reference:

```
# Shell commands (run in terminal, not Python)
pwd
ls
cd Documents
python hello.py
```

What it shows: Navigate folders then run a Python file from the correct directory.

Topic guide

What it actually is

The command line is a text interface to the operating system - you type commands instead of using the mouse.

When to use it
Running scripts, installing packages, git, servers, automation, following dev tutorials.

Where to use it
Windows Terminal, Mac Terminal, Linux SSH sessions, cloud VM admin.

Real use example
grep 'ERROR' app.log | tail -20 shows the last 20 error lines from a large server log.

Chapter 2: Managing Files from the Terminal

Chapter focus: Managing Files from the Terminal

Use pathlib.Path for modern path handling. Check Path('data.csv').exists() before reading. Encoding='utf-8' avoids character errors on Windows.

Code Reference:

```
with open("notes.txt", "w") as f:
    f.write("Line 1\nLine 2\n")

with open("notes.txt") as f:
    print(f.read())
```

What it shows: First block writes two lines; second reads the whole file back.

Topic guide

What it actually is
File handling is reading from or writing to disk. It connects your program's memory to persistent storage.

When to use it
Use files to save logs, load configs, export reports, read CSV datasets, or store user progress.

Where to use it
Data pipelines, backup scripts, game save files, reading homework datasets, generating invoices.

Real use example
A backup script copies every .docx from ~/Documents to ~/Backup using shutil and Path.glob('*.docx').

Chapter 3: Useful Command-Line Tools

Chapter focus: Useful Command-Line Tools

The command line (terminal) lets you control the computer with text commands. pwd shows where you are; ls lists files; cd changes folder; python script.py runs Python. Many tutorials and servers assume terminal use. It is faster than clicking for batch tasks and required for pip, git, and deployment.

Code Reference:

```
# Shell commands (run in terminal, not Python)
```

```
pwd
ls
cd Documents
python hello.py
```

What it shows: Navigate folders then run a Python file from the correct directory.

Topic guide

What it actually is
The command line is a text interface to the operating system - you type commands instead of using the mouse.

When to use it
Running scripts, installing packages, git, servers, automation, following dev tutorials.

Where to use it
Windows Terminal, Mac Terminal, Linux SSH sessions, cloud VM admin.

Real use example
A developer cd into their project, runs python -m pytest, and sees test results in seconds without opening an IDE.

Chapter 4: Python Setup and First Program

Chapter focus: Python Setup and First Program

Before writing Python programs, you install the Python interpreter from python.org (choose Python 3). During setup on Windows, check 'Add Python to PATH' so you can run python from the terminal. After installation, open a terminal and type python --version to confirm. You can write code in any text editor, but beginners often use IDLE (included with Python) or VS Code. The interactive shell (>>> prompt) is useful for testing one line at a time; saved .py files are for complete programs.

Code Reference:

```
import sys
print(sys.version)
print("Setup OK")
```

What it shows: import sys loads system info; sys.version shows your Python version.

Topic guide

What it actually is
Installation means putting the Python interpreter and tools on your computer so it can read and execute .py files. PATH is a list of folders Windows/Mac search when you type a command.

When to use it
Install Python once on any machine where you plan to write or run Python code. Reinstall or upgrade when you need a newer version for a library or course.

Where to use it
On your laptop for learning, on a server for web apps, on a Raspberry Pi for hardware projects, or in cloud notebooks like Google Colab (no local install needed there).

Real use example
A student installs Python 3.12, opens VS Code, creates hello.py, runs it from the terminal, and sees 'Setup OK' - confirming the environment works before starting homework.

Chapter 5: Variables, Data Types, and Control Flow

Chapter focus: Variables, Data Types, and Control Flow

A variable is a name that points to a value stored in memory. You create one with the assignment operator (=): the name goes on the left, the value on the right. Python figures out the type automatically - you do not declare int or str first. Common types are integers (whole numbers), floats (decimals), strings (text in quotes), and booleans (True/False). You can change a variable later by assigning a new value. Clear names like user_age or total_price make code easier to read than single letters.

Code Reference:

```
count = 5          # integer
label = "box"      # string
price = 2.5        # float
is_open = True     # boolean
print(type(count))  # <class 'int'>
```

What it shows: Each variable stores one value. type() tells you what kind of data it holds.

Topic guide

What it actually is
Variables are labeled containers for data. The label (name) lets you reuse and update values throughout a program without repeating the actual number or text.

When to use it
Use variables whenever a value might change, will be reused later, or needs a meaningful name - scores, user names, prices, flags, counters.

Where to use it
Every Python program: games (player score), web apps (username), data scripts (file path), calculators (operands), and loops (counters).

Real use example
An online shop stores item_price = 499, quantity = 2, and computes total = item_price * quantity so the checkout page shows Rs 998 without hard-coding numbers.

Chapter 6: Functions and Combining Shell with Python

Chapter focus: Functions and Combining Shell with Python

Keep functions small and named after what they do. Document tricky ones with a one-line docstring. Return early when possible to avoid deep nesting.

Code Reference:

```
def area(width, height):
    return width * height

def print_area(w, h):
    print(area(w, h))

print_area(4, 5)  # 20
```

*What it shows: area computes and returns; print_area reuses area instead of duplicating w * h.*

Topic guide

What it actually is
A function is a named, reusable mini-program inside your program. You call it by name with arguments; it may return a result.

When to use it
Use a function when the same logic appears twice, when a task has a clear name, or when you want to test one piece separately.

Where to use it
Calculations, formatting output, validating input, API handlers, and splitting large scripts into readable pieces.

Real use example
A password checker function validate(pw) returns True/False and is reused on signup, login, and reset forms.